FinVeda Project Master Documentation

Project: FinVeda - AI-Powered Financial Intelligence Platform

Version: 1.0

Date: January 25, 2026

Author: Professional Technology Architect

PART 1: Business Requirements Document (BRD)

1. Executive Summary

FinVeda is designed to bridge the gap between complex market data and actionable financial decisions for retail investors. Unlike traditional trading platforms that focus on execution, FinVeda focuses on **intelligence**—using AI to analyze portfolios, optimize tax liabilities, and assess risk behavior. This document outlines the business needs to transition the current high-fidelity prototype into a live, scalable commercial product.

2. Business Objectives

- **Democratize Financial Intelligence:** Provide institutional-grade analytics to retail investors without the high cost of human advisory.
- **Increase User Financial Literacy:** Integrate education directly into the investment workflow to improve long-term user retention.
- **Data-Driven Wealth Management:** Shift user behavior from emotional trading to goal-aligned, data-backed investing.

3. Target Audience

- **Primary:** Indian retail investors (Ages 25-45) who are tech-savvy but seek guidance on asset allocation and risk.
- **Secondary:** Early-stage investors looking for structured learning and automated portfolio tracking.

4. Project Scope

4.1 In-Scope (MVP Phase)

- User Authentication & Secure Onboarding.
- Dashboard for Portfolio Visualization (Holdings, Allocation).

- AI-Driven Risk Profiling Questionnaire & Scoring.
- Basic Tax Optimization Suggestions (Logic-based).
- Educational Content Hub (Articles, Courses).
- Market News Feed Integration.

4.2 Out-of-Scope (Phase 1)

- Direct Trade Execution (Buying/Selling stocks directly on platform).
- Cryptocurrency Wallet Integration.
- Social Trading / Community Features.

5. Functional Requirements (Business Perspective)

ID	Requirement Area	Description	Priority
BR-01	**User Onboarding**	Users must be able to create an account securely and link their financial goals to their profile.	High
BR-02	**Portfolio Tracking**	The system must visualize current asset distribution and performance against benchmarks.	High
BR-03	**Risk Intelligence**	The platform must assign a dynamic "Risk Score" based on user behavior and inputs, not just a static survey.	Critical
BR-04	**Tax Optimization**	The system must identify potential tax-saving opportunities (e.g., harvesting losses) based on Indian tax laws.	Medium

BR-05	**Education**	Users should receive content recommendations based on their portfolio composition (e.g., ETF articles if they hold ETFs).	Medium

6. Non-Functional Requirements

- **Security:** All user data must be encrypted at rest and in transit. Compliance with local financial data regulations is mandatory.
- **Performance:** Dashboard load times must be under 2 seconds to ensure a seamless experience.
- **Scalability:** The system must support concurrent access for at least 10,000 active users initially.

PART 2: Software Requirements Specification (SRS)

1. Introduction

1.1 Purpose

The purpose of this document is to define the functional and non-functional requirements for the FinVeda platform. It serves as the primary guide for the development and QA teams.

1.2 Tech Stack

- **Frontend:** React (TypeScript) + Tailwind CSS
- **Backend:** Python FastAPI
- **Database:** PostgreSQL

2. User Personas

- **Rohan (The Novice):** Needs guidance, simple dashboards, and educational jargon-busting.
- **Priya (The Strategist):** Needs detailed risk metrics, tax harvesting reports, and goal tracking.
- **Admin:** Needs to manage content, view system health, and manage user roles.

3. System Features & Functional Requirements

3.1 User Authentication Module

- **FR-01:** System shall support email/password registration and Google OAuth.
- **FR-02:** System shall implement Multi-Factor Authentication (MFA) for profile setting changes.
- **FR-03:** Password policies must enforce complexity (min 12 chars, alphanumeric).

3.2 Portfolio Dashboard

- **FR-04:** System shall display a consolidated view of Total Asset Value.
- **FR-05:** System shall render interactive charts (Pie/Line) for Asset Allocation and Historic Performance using Recharts or similar libraries.
- **FR-06:** Users shall be able to manually add assets or upload CSV statements (MVP).

3.3 AI Intelligence Engine

- **FR-07 (Risk Profiling):** The backend shall calculate a Risk Score (1-100) based on user questionnaire responses and volatility of added assets.
- **FR-08 (Tax Engine):** The system shall analyze holdings to flag Long Term Capital Gains (LTCG) vs Short Term (STCG) implications.

3.4 Education & Content

- **FR-09:** The system shall present a "Recommended for You" section based on user's portfolio tags.
- **FR-10:** Admins shall be able to publish articles via a Markdown-supported CMS interface.

4. External Interface Requirements

- **User Interface:** Responsive Web Design (Mobile-First) using Tailwind CSS.
- **API Interfaces:** RESTful APIs for client-server communication.
- **Market Data:** Integration with third-party providers (e.g., AlphaVantage, Yahoo Finance) for delayed market quotes.

5. Non-Functional Requirements

- **NFR-01 (Reliability):** 99.9% uptime during market hours.
- **NFR-02 (Latency):** API response time < 200ms for read operations.
- **NFR-03 (Security):** JWT (JSON Web Token) implementation for session management; 256-bit encryption for sensitive fields in DB.

PART 3: Technical Design Document (TDD)

1. System Overview

FinVeda utilizes a decoupled architecture. The frontend is a Single Page Application (SPA) built with React, communicating via REST APIs with a Python FastAPI backend. Data persistence is handled by PostgreSQL.

2. System Architecture

2.1 High-Level Diagram

- **Client Layer:** React.js + Tailwind CSS (Hosted on CDN/Edge).
- **API Gateway:** Nginx (Reverse Proxy & SSL Termination).
- **Application Layer:** Python FastAPI (Running in Docker Containers).
- **Data Layer:** PostgreSQL (Primary DB) + Redis (Caching Layer).

2.2 Technology Choices Rationale

- **FastAPI:** Chosen for native async support (critical for fetching market data) and easy integration with Python data science libraries (Pandas/NumPy) for the "Intelligence" features.
- **PostgreSQL:** Chosen for ACID compliance and robust support for complex financial queries.
- **Tailwind CSS:** Chosen for rapid UI development and design consistency.

3. Data Design (Schema)

3.1 Core Entities (ER Diagram Description)

Users Table

- id (UUID, PK)
- email (VARCHAR, Unique)
- password_hash (VARCHAR)
- created_at (TIMESTAMP)

Portfolios Table

- id (UUID, PK)
- user_id (UUID, FK -> Users.id)
- portfolio_name (VARCHAR)
- risk_score (INT)

Assets Table

- id (UUID, PK)
- ticker_symbol (VARCHAR)
- asset_type (ENUM: 'EQUITY', 'MF', 'GOLD')

- current_price (DECIMAL)

Transactions Table

- id (UUID, PK)
- portfolio_id (FK)
- asset_id (FK)
- type (ENUM: 'BUY', 'SELL')
- quantity (DECIMAL)
- price_per_unit (DECIMAL)
- transaction_date (TIMESTAMP)

4. Component Design

4.1 Frontend Components (React)

- AuthProvider: Manages global authentication state/tokens.
- DashboardLayout: Main wrapper with Sidebar and Topbar.
- PortfolioGraph: Reusable chart component utilizing generic data props.
- RiskMeter: Visual gauge component for risk scores.

4.2 Backend Modules (Python)

- auth_service: Handles Login, Signup, JWT generation.
- portfolio_manager: CRUD operations for portfolios and transactions.
- market_data_service: Adapter pattern to fetch prices from external APIs.
- analytics_engine: Contains the logic for Risk Scoring and Tax calculations (e.g., using Pandas DataFrames).

5. API Design (REST)

Method	Endpoint	Description
POST	/api/v1/auth/login	Authenticate user & return JWT.
GET	/api/v1/portfolio/summary	Get aggregated portfolio stats.
POST	/api/v1/portfolio/transaction	Record a buy/sell event.
GET	/api/v1/analytics/risk-profile	Trigger AI risk assessment.

6. Security Architecture

- **Transport:** TLS 1.3 enforced.
- **Authentication:** OAuth2 with Bearer Tokens (JWT).
- **Input Validation:** Pydantic models in FastAPI to strictly validate all incoming payloads.
- **Rate Limiting:** Implemented via Redis to prevent API abuse.

7. Deployment Strategy

- **Containerization:** Dockerfiles for both Frontend (multi-stage build) and Backend.
- **Orchestration:** Docker Compose for local dev; AWS ECS/Fargate for production.
- **CI/CD:** GitHub Actions pipeline to run tests (pytest, jest) and deploy on merge to main.